TP 4 - Création d'un Replica Set des serveurs de configuration et mise en place du sharding dans MongoDB

L'objectif de ce TP est de configurer un cluster sharded avec MongoDB, en mettant en place des serveurs de configuration et des réplicas de shards pour un environnement de production scalable.

Introduction

Dans MongoDB, le **sharding** est une technique permettant de répartir les données sur plusieurs machines pour améliorer les performances et la scalabilité. Afin de configurer un cluster sharded, MongoDB utilise des serveurs de configuration qui gèrent les métadonnées du cluster, et des **shards** qui contiennent les données partitionnées. Le cluster comprend également un ou plusieurs **mongos** (routers) qui acheminent les requêtes vers les bons shards.

Dans cette configuration, il est essentiel d'avoir **deux serveurs de configuration** pour garantir la disponibilité et la tolérance aux pannes. Ces serveurs de configuration sont organisés en un **Replica Set** et gèrent les métadonnées des shards. Les shards eux-mêmes peuvent être organisés en réplicas pour garantir la disponibilité des données.

Étape 1 : Création de trois serveurs de configuration (Config Servers) avec leur espace de stockage

1. **Création des répertoires pour chaque serveur MongoDB** :
   - Création d'un nouveau dossier nommé **configdb1** dans le répertoire **C:\DATA**
   - Création d'un nouveau dossier nommé **configdb2** dans le répertoire **C:\DATA**

2. **Démarrer les serveurs de configuration** :

```
mongod --configsvr --replSet ConfigRS --port 27019  -- dbpath  C:\DATA\configdb1
```

et :

```
mongod --configsvr --replSet ConfigRS --port 27020 --dbpath C:\DATA\configdb2
```

3. **Initialisation du Replica Set ConfigRS** : **mongo --port 27019**
   - Initialisez le Replica Set :
     rs.initiate()
     rs.add("DESKTOP-VVH4030:27020") ou bien rs.add("localhost:27020")

     rs.status()

Étape 2 : Création des serveurs de Shards

1. **Création des répertoires pour chaque serveur de Sharding** :
   - Création d'un nouveau dossier nommé **shard1** dans le répertoire **C:\DATA**
   - Création d'un nouveau dossier nommé **shard2** dans le répertoire **C:\DATA**

2. **Démarrer les serveurs de shard** :

```
mongod --shardsvr --replSet sh1 --port 27031 --dbpath /data/shard1
```

```
mongod --shardsvr --replSet sh2 --port 27032 --dbpath /data/shard2
```

- **Initialisation du Replica Set de chaque shard (sh1 et sh2)**:

- Connectez-vous à l'un des serveurs de configuration :
  mongo --port 27031
- Initialisez le Replica Set :
  rs.initiate()
- Connectez-vous à l'un des serveurs de configuration :
  mongo --port 27032
- Initialisez le Replica Set :
  rs.initiate()

3. **Démarrer mongos (le routeur de requêtes)** :

```
mongos --configdb ConfigRS/DESKTOP-VVH4030:27019 --port 27018
```

ou bien

```
mongos --configdb ConfigRS/localhost:27019 --port 27018
```

4. **Ajouter les shards au cluster** : une fois mongos démarré, nous devons ajouter les shards au cluster sharded.
   - Se connecter à mongos
     Mongo –port 27018
   - Ajouter les shards au cluster

bien sh.addShard("sh1/localhost:27031")

sh.addShard("sh2/localhost:27032")

Étape 3 : Sharding de la base de données et insertion de données

1. **Activer le sharding pour la base de données** : Avant de sharder des collections, vous devez activer le sharding pour la base de données en utilisant la commande sh.enableSharding().

```
mongos> use testDB
mongos> sh.enableSharding("testDB")
mongos> db.createCollection("testCollection") ;
mongos> db.test.createIndex({"_id": 1});
mongos> sh.shardCollection("testDB.testCollection",{"_id" :1}) ;
```

- enableSharding("testDB")→ permet d'actives le sharding pour la base de données testDB. Cela signifie que toutes les collections créées dans cette base de données pourront être shardées, et MongoDB commencera à gérer la distribution des données entre les shards.
- `sh.shardCollection("testDB.testCollection", {"_id": 1});:` cette commande shard la collection `testCollection` en utilisant le champ `_id` comme clé de sharding. MongoDB partitionne les documents dans la collection en fonction des valeurs de ce champ `_id` et répartit les données entre les différents shards du cluster

```
mongoimport --db testDB --collection testCollection --port 27018 --file testdata.json
```

Test 2 :

```
var bulkInsert = db.testCcollection.initializeUnorderedBulkOp();
names = ["Mohaned", "Samiza", "All", "Hassan", "Moriem", "Sami", "Rachid", "Sara", "Karim","Karima",
"Jamal" ]
var bulkInsert = db.testCollection.collection.initializeUnorderedBulkOp(); // Corrected the bulk operation
initialization
```

```
// Array of names
var names = ["Mohaned", "Samiza", "All", "Hassan", "Moriem", "Sami", "Rachid", "Sara", "Karim",
"Karima", "Jamal"];
for (var i = 0; i < 20000003; i++) {
    var userId = i;
    var name = names[Math.floor(Math.random() * names.length)];
    var number = Math.floor(Math.random() * 10001); // Generates random numbers from 0 to 10000
   bulkInsert.insert({
        userId: userId,
        name: name,
        number: number
    });

    if (i % 1000 === 0) {
        bulkInsert.execute();
        bulkInsert = db.testCollection.initializeUnorderedBulkOp(); // Reinitialize the bulk operation
    }
}

bulkInsert.execute();
```